threads. The following listing shows the amended `pop()` function, and listing 7.5 shows the supporting functions for such an implementation.

Listing 7.4. Reclaiming nodes when no threads are in `pop()`

```
template<typename T>
class lock_free_stack
{
private:
    std::atomic<unsigned> threads_in_pop;
    void try_reclaim(node* old_head);
public:
    std::shared_ptr<T> pop()
    {
        ++threads_in_pop;
        node* old_head=head.load();
        while(old_head &&
              !head.compare_exchange_weak(old_head,old_head->next));
        std::shared_ptr<T> res;
        if(old_head)
        {
            res.swap(old_head->data);
        }
        try_reclaim(old_head);
        return res;
    }
};
```

1 Atomic variable

2 Increase counter before doing anything else

3 Reclaim deleted nodes if you can

4 Extract data from node rather than copying pointer

The atomic variable `threads_in_pop` ❶ is used to count the threads currently trying to pop an item off the stack. It's incremented at the start of `pop()` ❷ and decremented inside `try_reclaim()`, which is called once the node has been removed ❹. Because you're going to potentially delay the deletion of the node itself, you can use `swap()` to remove the data from the node ❸ rather than just copying the pointer, so that the data will be deleted automatically when you no longer need it rather than it being kept alive because there's still a reference in a not-yet-deleted node. The next listing shows what goes into `try_reclaim()`.

Listing 7.5. The reference-counted reclamation machinery